

React Router V6

What is React Router?

React Router is a library that provides navigational components for React developers to create Single-Page Applications (SPAs) with dynamic, client-side routing.

Applications that use React-Router can benefit from the separation of content afforded to multi-page applications without the break in the user-experience caused by page reloads.

React <RouterProvider>

React Router can be provided to the entire application using the `<RouterProvider>` component (from `react-router-dom`) and including a `router` attribute. In the given example React Router is provided to the entire application using a `<RouterProvider>` component in the main `App` component.

```
import { RouterProvider } from 'react-router-dom';

function App() {
  return (
    <RouterProvider router={ /* initialized router object */ } />
  );
}

export default App;
```

React Router <Route>

React Router uses routes to decide what components to render based on the URL path. The `<Router />` component (from `react-router-dom`) is used to define routes which will render the given `element` if the current URL matches the given `path` prop. In the provided example a `<Route>` is created that renders the `<Home/>` component when the URL path matches `/home`.

```
import { Route } from 'react-router-dom';
import Home from './Home';

<Route path='/home' element={ <Home/> } />
```

Link

React Router's `<Link>` component can be used to create links for navigation. The `to` prop specifies the location to which the user will be redirected after clicking on the `<Link>`. Rendering a `<Link>` will insert an anchor tag (`<a>`) in your HTML document, but the anchor's default behavior (triggering a page reload) will be disabled. This allows the application's `<Router>` to respond to URL changes by rendering the appropriate content.

```
<Link to="/about">About</Link>
```

React Router <NavLink>

The React Router `<NavLink to=">` component (from `react-router-dom`) is a special type of `<Link>` that can be styled differently when the link is active, that is, it's `to` prop value matches the current URL. A function can be passed to `className` or `style` attributes to apply a custom "active" style or class. In the given example a function is passed to the `NavLink` `className` where the `isActive` property is de-structured and used to apply the `active-navlink` class when `isActive` is `true`.

```
import { NavLink } from `react-router-dom`;

function Navigation() {
  return(
    <NavLink
      to="/home"
      className={ ({ isActive }) => isActive ? 'active-
navlink': ''}>
      Home
    </NavLink>
  );
}

export default Navigation;
```

URL Parameters

URL parameters are dynamic (ie. non-constant) segments of a `<Route>` component's `path` prop. They can be used to dynamically serve resources based on the current window location.

A URL parameter begins with a colon and is followed by the name of the parameter, like so: `:parameter`. To specify that a URL parameter is optional, append a question mark, like so: `:parameter?`.

```
import { BrowserRouter as Router, Route } from "react-router-dom"
import Book from "../features/books/Book"

function App () {
  return (
    <Router>
      { /* bookId is required to render <Book /> */ }
      { /* page is not required to render <Book /> */ }
      <Route path="/books/:bookId/:page?">
        <Book />
      </Route>
    </Router>
  )
}
```

useParams()

React Router's `useParams()` hook can be used by a component rendered by a `<Route>` with a dynamic path to get the names and values of the current URL's parameters.

This function returns an object containing a key/value pair for each URL parameter where the key is the URL parameter's name and the value is the parameter's current value.

```
import React from "react";
import { useParams } from "react-router-dom";
```

```
// assume this component is rendered by a <Route> with
the path "/users/:userName"
```

```
export default const UserProfile () {
  const { userName } = useParams()
  return (
    <h1> Welcome {userName}! </h1>
  )
  /*
```

If the user visits `/users/Codey`, the following will be rendered:

```
<h1> Welcome Codey!
*/
}
```

React Router <Navigate>

The React Router `<Navigate to=""/code> component (from react-router-dom) is used to declaratively navigate to the URL path specified by the to prop. In the given example a user is declaratively redirected to the /home URL path if they are not authenticated.`

```
import { Navigate } from 'react-router-dom';

function Profile({ isAuthenticated }) {
  if (!isAuthenticated) {
    <Navigate to='/home' />
  }

  return (
    <h1>
      Welcome
    </h1>
  );
}

export default Profile;
```

React Router `useNavigate()`

The React Router `useNavigate()` hook (from `react-router-dom`) is used to imperatively redirect to a specified URL path or integer delta in the history stack.

The given example renders 3 buttons:

- Home - which navigates to the URL path `/home`
- Back - which navigates 1 URL path back in the history stack
- Forward - which navigates 1 URL path forward in the history stack.

```
import { useNavigate } from 'react-router-dom';

function Actions() {
  const navigate = useNavigate();

  function goHome() {
    navigate("/home"); // navigate to exact path
  }

  function goBack() {
    navigate(-1); // navigate 1 URL path back in the
    history stack
  }

  function goForward() {
    navigate(1); // navigate 1 URL path forward in the
    history stack
  }

  return (
    <div>
      <button onClick={ goHome }>Home</button>
      <button onClick={ goBack }>Back</button>
      <button onClick={ goForward }>Forward</button>
    </div>
  );
}
```

```
}
```

```
export default Actions;
```

Query Parameters

Query parameters appear in URLs beginning with a question mark (?) and are followed by a parameter name assigned to a value. They are optional and are most often used to search, sort and/or filter resources.

For example, if you were to visit the provided URL you would be taken to Google's /search page displaying results for the search term 'codecademy'. In this example, the name of the query parameter is q .

```
https://www.google.com/search?q=codecademy
```

Creating A React Router

The React Router utility methods `createBrowserRouter()` and `createRoutesFromElements()` (from `react-router-dom`) are used to create and initialize a router object using JSX routes. `createBrowserRouter()` is used to create a router object by passing a list of route objects. `createRoutesFromElements()` is used to turn JSX routes (created using `<Route>`) into route objects.

The given example defines and initializes an object called `appRouter` with React Router routes using `createBrowserRouter()` and `createRoutesFromElements()` .

```
import { createBrowserRouter, createRoutesFromElements,  
Route } from 'react-router-dom';
```

```
const appRouter =  
  createBrowserRouter(createRoutesFromElements(  
    <Route path="/" element={ <Home/> }/>  
  ));
```


React Router Nested Routes

In React Router `<Route>` components can be nested within other `<Route>` components by enclosing them in their opening and closing tags. A `<Route>` that contains other `<Route>` components is known as the parent route and a `<Route>` within another `<Route>` is called the child route. The child routes `path` attribute is relative to its parents.

The given example creates a parent route on the root path `/` and a couple of nested child routes with the paths `/home` and `/about`.

```
import { createBrowserRouter, createRoutesFromElements,
Route } from 'react-router-dom';

const appRouter =
  createBrowserRouter(createRoutesFromElements(
    <Route path='/' element={ <Root/> }> { /* parent route
    */ }

      <Route path='home' element={ <Home/> }/> { /*
    child route with path relative to parent ('/') */}

      <Route path='about' element={ <About/> }/> { /* child
    route with path relative to parent ('/') */}

    </Route>
  ));
```

React Router <Outlet>

The React Router `<Outlet/>` component (from `react-router-dom`) is used within the parent route element to indicate where a child route element should be rendered.

The given example creates a `<Route>` at the root path `/` that renders an element `<Root/>` and a child route on the path `/home`. The `<Outlet/>` component is used within `<Root/>` to specify where in the view the child element, `<Home />`, should be rendered.

```
/* start App.js */
import { BrowserRouter, createRoutesFromElements,
Route } from 'react-router-dom';
// components imports ...

const appRouter =
  BrowserRouter(createRoutesFromElements(
    <Route path='/' element={ <Root/> }>
      <Route path='home' element={ <Home/> }/>
    </Route>
  ));

// rest of code ...

/* end App.js */

/* start Root.js */
import { Outlet } from 'react-router-dom';

function Root() {
  return (
    <>
      <h1>Welcome</>
      <Outlet/> { /* indicates where <Home/> should be
rendered within <Root> */}
    </>
  );
}
```

```
    );  
  
  }  
  
  export default Root;
```

React Router Index Route

When creating nested routes in React Router, a default child route can be rendered on its parent's path using the `index` attribute on the child `<Route>` and omitting `path /`.

The given example creates a parent route on the root path `/` and an index child `<Route>` that renders `<Home/>` on the path `/`.

```
import { createBrowserRouter, createRoutesFromElements,  
Route } from 'react-router-dom';  
  
// component imports  
  
const appRouter =  
  createBrowserRouter(createRoutesFromElements(  
    <Route path='/' element={ <Root/> }>  
      <Route index element={ <Home/> }/> { /* renders on  
`/*` */}  
    </Route>  
  ));  
  
// rest of code ...
```

React Router `useSearchParams()`

The React Router `useSearchParams()` hook (from `react-router-dom`) returns an object (typically called `searchParams`) which contains the current URL location's query parameters and a function (typically called `setSearchParams`) to update the query params. Query parameters values can be retrieved using the `searchParams.get()` method and passing a string with the name.

The given example gets the values from the `name` and `age` query parameters and displays them in the component.

```
import { useSearchParams } from 'react-router-dom';

function Profile() {
  const [ searchParams, useSearchParams ] =
    useSearchParams(); // get object containing query
    parameters

  // retrieve query parameter values
  const name = searchParams.get('name');
  const age = searchParams.get('age');

  return (
    <div>
      <h1>Name: { name }</h1>
      <h2>Age: { age }</h2>
    </div>
  );
}

export default Profile;
```

